E-Commerce Microservices

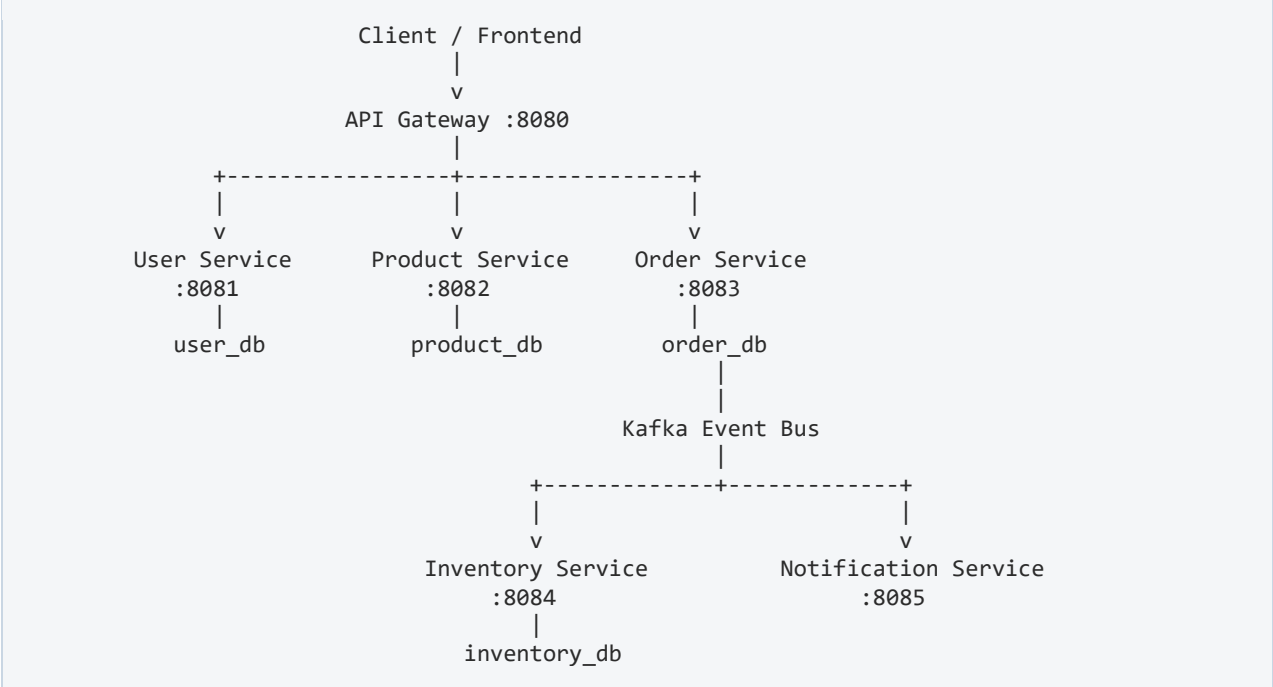
Complete Development Guide

Architecture • Services • Kafka Events • DevOps Pipeline

Table of Contents

1. Final Architecture

The system follows a client-facing API Gateway that routes requests to three core domain services (User, Product, Order). The Order Service publishes events onto a Kafka event bus, which is consumed asynchronously by the Inventory and Notification services.



2. Microservices List

The system is composed of six independent microservices:

Service	Port	Main Responsibility
api-gateway	8080	External requests / routing
user-service	8081	Register, Login, JWT, Users
product-service	8082	Product management
order-service	8083	Orders
inventory-service	8084	Stock management
notification-service	8085	Notifications

Communication Pattern

```
User Service        --> REST
```

```
Product Service      --> REST
Order Service         --> REST + Kafka Producer
Inventory Service    --> Kafka Consumer
Notification Service --> Kafka Consumer
```

3. Development Order

Follow this build sequence from application development through to full DevOps deployment:

```
1. User Service
2. Product Service
3. Order Service
4. Kafka Integration
5. Inventory Service
6. Notification Service
7. API Gateway
8. Docker
9. Docker Compose
10. GitHub Actions
11. Terraform
12. AWS
13. Kubernetes
14. Ansible
15. CloudWatch
```

Part A — User Service

4. User Service Responsibility

```
user-service
├── Register
├── Login
├── JWT Generation
├── JWT Validation
├── Get Current User
├── Get User
├── Update User
└── Role Authorization
```

Roles: CUSTOMER, ADMIN

5. User Service Package Structure

```
user-service/
└── src/main/java/com/ecommerce/userservice/
    ├── controller/
    │   ├── AuthController.java
    │   └── UserController.java
    ├── service/
    │   ├── AuthService.java
    │   └── UserService.java
    ├── repository/
    │   └── UserRepository.java
    ├── entity/
    │   ├── User.java
    │   └── Role.java
    ├── dto/
    │   ├── RegisterRequest.java
    │   ├── LoginRequest.java
    │   ├── AuthResponse.java
    │   └── UserResponse.java
    ├── security/
    │   ├── SecurityConfig.java
    │   ├── JwtService.java
    │   └── JwtAuthenticationFilter.java
    └── exception/
        ├── GlobalExceptionHandler.java
        └── ResourceNotFoundException.java
```

Part B — Product Service

Development begins once the User Service is complete.

6. Product Service Responsibility

```
product-service
├── Create Product
├── Get Products
├── Get Product by ID
├── Update Product
└── Delete Product
```

Product Entity

```
Product
├── id
├── name
├── description
├── price
├── category
├── quantity
├── createdAt
└── updatedAt
```

Database

```
product-service --> PostgreSQL --> product_db --> products
```

7. Product Package Structure

```
product-service/
└── src/main/java/com/ecommerce/productservice/
    ├── controller/
    │   └── ProductController.java
    ├── service/
    │   └── ProductService.java
    ├── repository/
    │   └── ProductRepository.java
    ├── entity/
    │   └── Product.java
    ├── dto/
    │   ├── ProductRequest.java
    │   └── ProductResponse.java
    ├── security/
    │   ├── SecurityConfig.java
    │   └── JwtAuthenticationFilter.java
    └── exception/
        └── GlobalExceptionHandler.java
```

8. Product APIs

Public

```
GET /api/products
GET /api/products/{id}
```

ADMIN only

```
POST   /api/products
PUT    /api/products/{id}
DELETE /api/products/{id}
```

Example request body:

```
{
  "name": "Laptop",
  "description": "Dell Laptop",
  "price": 250000,
  "category": "Electronics",
  "quantity": 20
}
```

Part C — Order Service

9. Order Service Responsibility

```
order-service
├── Create Order
├── Get Order
├── Get User Orders
├── Cancel Order
└── Publish Order Events
```

This is where Kafka is introduced into the architecture.

10. Order Data Model

```
Order
├── id
├── userId
├── totalAmount
├── status
├── createdAt
└── updatedAt

OrderItem
├── id
├── orderId
├── productId
├── quantity
└── price
```

Order Status

```
PENDING
CONFIRMED
CANCELLED
```

11. Order Flow

```
Select Product
      ↓
Create Order
      ↓
API Gateway
      ↓
Order Service
      ↓
Validate User
      ↓
Validate Product
      ↓
Create Order
      ↓
Publish Kafka Event
```

12. Order APIs

```
POST /api/orders
GET  /api/orders
GET  /api/orders/{id}
PUT  /api/orders/{id}/cancel
```

Example request body:

```
{
  "items": [
    { "productId": 10, "quantity": 2 }
  ]
}
```

Part D — Kafka (Event-Driven Microservices)

13. Why Kafka?

Direct REST calls create tight coupling between services:

```
Order Service --HTTP--> Inventory Service
```

Kafka decouples the producer from its consumers:

```
Order Service
      |
      | Event
      v
    Kafka
      |
      +-------------> Inventory Service
      |
      +-------------> Notification Service
```

This allows the Order Service to publish an event without being tightly dependent on the Inventory or Notification services.

14. Kafka Topics

Three topics are used:

```
order-created
order-cancelled
stock-updated
```

order-created

```
Order Service --> order-created --> Inventory Service
                                --> Notification Service
```

order-cancelled

```
Order Service --> order-cancelled --> Inventory Service
                                  --> Notification Service
```

stock-updated

```
Inventory Service --> stock-updated --> Notification Service
```

15. Kafka Producer (Order Service)

```
Order created --> OrderService --> KafkaProducer --> order-created
```

Example event payload:

```
{
  "eventType": "ORDER_CREATED",
  "orderId": 1001,
  "userId": 5,
  "items": [
    { "productId": 10, "quantity": 2 }
  ]
}
```

16. Kafka Consumers

Inventory Service

```
Kafka --> order-created --> Inventory Consumer --> Check Stock --> Reduce Stock
```

Example: Laptop stock = 20, customer orders 2, remaining stock = 18.

Notification Service

```
Kafka --> order-created --> Notification Consumer --> Create notification
```

Example: "Order #1001 created successfully"

Part E — Inventory Service

17. Inventory Entity

```
Inventory
├── id
├── productId
├── availableQuantity
├── reservedQuantity
└── updatedAt
```

Database: inventory_db → inventory

18. Inventory Service Flow

```
Order Service
      |
      v
     Kafka
      |
      v
Inventory Service
      |
      v
 Check Stock
      |
   +--+--+
   |     |
Enough  Not enough
   |     |
   v     v
Reduce  Reject
Stock   / Event
```

Part F — Notification Service

Kept intentionally simple in the first iteration.

```
Kafka --> Notification Service --> Consume Event --> Log / Notification
```

Future extensions (add later, not in the first build):

- Email
- SMS
- Push Notification

Keeping this service minimal initially avoids adding unnecessary application complexity, since the main focus of this phase is DevOps.

Part G — API Gateway

20. API Gateway

External clients never connect directly to individual services.

```
Frontend
   |
   v
API Gateway :8080
   |
   +----> User Service :8081
   |
   +----> Product Service :8082
   |
   +----> Order Service :8083
```

Gateway Routes

```
/api/auth/**       -> user-service
/api/users/**      -> user-service
/api/products/**   -> product-service
/api/orders/**     -> order-service
```

21. JWT + Gateway

```
Client
  |
  | Authorization: Bearer JWT
  v
API Gateway
  |
  | Validate JWT
  v
Microservice
```

The Gateway alone does not form the complete security boundary. Each microservice also performs its own authorization checks — a Defense-in-Depth approach.

Part H — Service-to-Service Communication

Synchronous

```
Order Service --REST--> Product Service
```

Use when an immediate response is required.

Asynchronous

```
Order Service
      |
      | Kafka Event
      v
     Kafka
      |
      +----> Inventory
      |
      +----> Notification
```

Use for background or event-driven processing that does not need an immediate response.

Part I — Configuration

Every service uses an application.yml file:

```
server:
  port: 8081

spring:
  application:
    name: user-service
  datasource:
    url: ${DB_URL}
    username: ${DB_USERNAME}
    password: ${DB_PASSWORD}

management:
  endpoints:
    web:
      exposure:
        include: health,info,metrics
```

Never hardcode secrets. Secret management strategy by environment:

Environment	Secret Source
Local	Environment Variables
Docker	Environment Variables
Kubernetes	Secrets
AWS	Secrets Manager

Part J — Testing

22. Per-Service Testing

Service	Test Coverage
User Service	Register, Login, JWT, Get User, Update User, Authorization
Product Service	Create, Read, Update, Delete
Order Service	Create Order, Get Order, Cancel Order
Kafka	Create Order → Kafka → Inventory receives event → Stock updated

23. Integration Test — Full Business Flow

```
1. Register
2. Login
3. Receive JWT
4. Get Products
5. Create Order
6. order-created Kafka event
7. Inventory reduces stock
8. Notification created
```

A successful run of this flow means the application development phase is essentially complete.

Part K — Actuator

Every service exposes:

```
GET /actuator/health
```

Response:

```
{ "status": "UP" }
```

Used later for Kubernetes probes:

```
Liveness  -> /actuator/health/liveness
Readiness -> /actuator/health/readiness
```

Part L — Docker

Docker work begins only after application development is complete.

```
user-service --> Dockerfile --> Docker Image --> Container
```

Every service gets its own Docker image:

- api-gateway
- user-service
- product-service
- order-service
- inventory-service
- notification-service

Part M — Docker Compose

Local complete environment definition:

```
Docker Compose
├── api-gateway
├── user-service
├── product-service
├── order-service
├── inventory-service
├── notification-service
├── user-db
├── product-db
├── order-db
├── inventory-db
└── Kafka
```

Starts the full system with a single command:

```
docker compose up -d
```

Part N — CI/CD

```
Developer
    |
    v
git push
    |
    v
GitHub Actions
    |
    +---- Build
    +---- Test
    +---- Docker Build
    +---- Security Scan
    +---- Push Image
    |
    v
Container Registry
```

In the AWS phase:

```
GitHub Actions --> Amazon ECR --> Kubernetes / EKS
```

Part O — Terraform

Terraform = Infrastructure as Code. It provisions:

```
Terraform
├── VPC
├── Subnets
├── Security Groups
├── EKS
├── ECR
├── RDS
├── IAM
└── Kafka / MSK
```

Core commands:

```
terraform init
terraform validate
terraform plan
terraform apply
```

Part P — Kubernetes

```
EKS Cluster
├── api-gateway
├── user-service
├── product-service
├── order-service
├── inventory-service
└── notification-service
```

Each service follows the pattern:

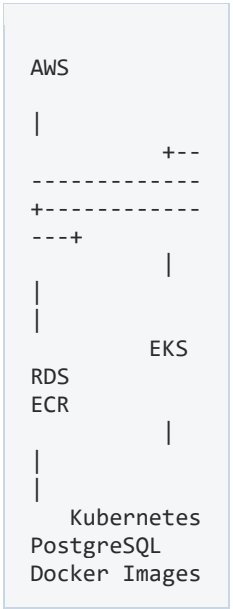
```
Deployment --> Pods --> Service
```

Resources implemented per service:

- Deployment
- Service
- ConfigMap
- Secret
- Ingress
- Liveness Probe
- Readiness Probe
- Resource Limits
- HPA (Horizontal Pod Autoscaler)
- Rolling Update

Part Q — AWS

Production architecture:



```
          |

Microservices
          |
         MSK
          |
        Kafka
```

AWS Service	Purpose
EKS	Kubernetes
ECR	Docker images
RDS	PostgreSQL
MSK	Managed Kafka
VPC	Network
IAM	Permissions
Load Balancer	External traffic
CloudWatch	Monitoring

Part R — Ansible

Terraform and Ansible serve different purposes:

Tool	Purpose
Terraform	Create Infrastructure
Ansible	Configure Infrastructure

```
Terraform -->
Create AWS
infrastructure -->
Ansible -->
Configure required
servers/environment
```

Note: managed AWS services such as EKS and MSK are not manually configured with Ansible.

Part S — CloudWatch

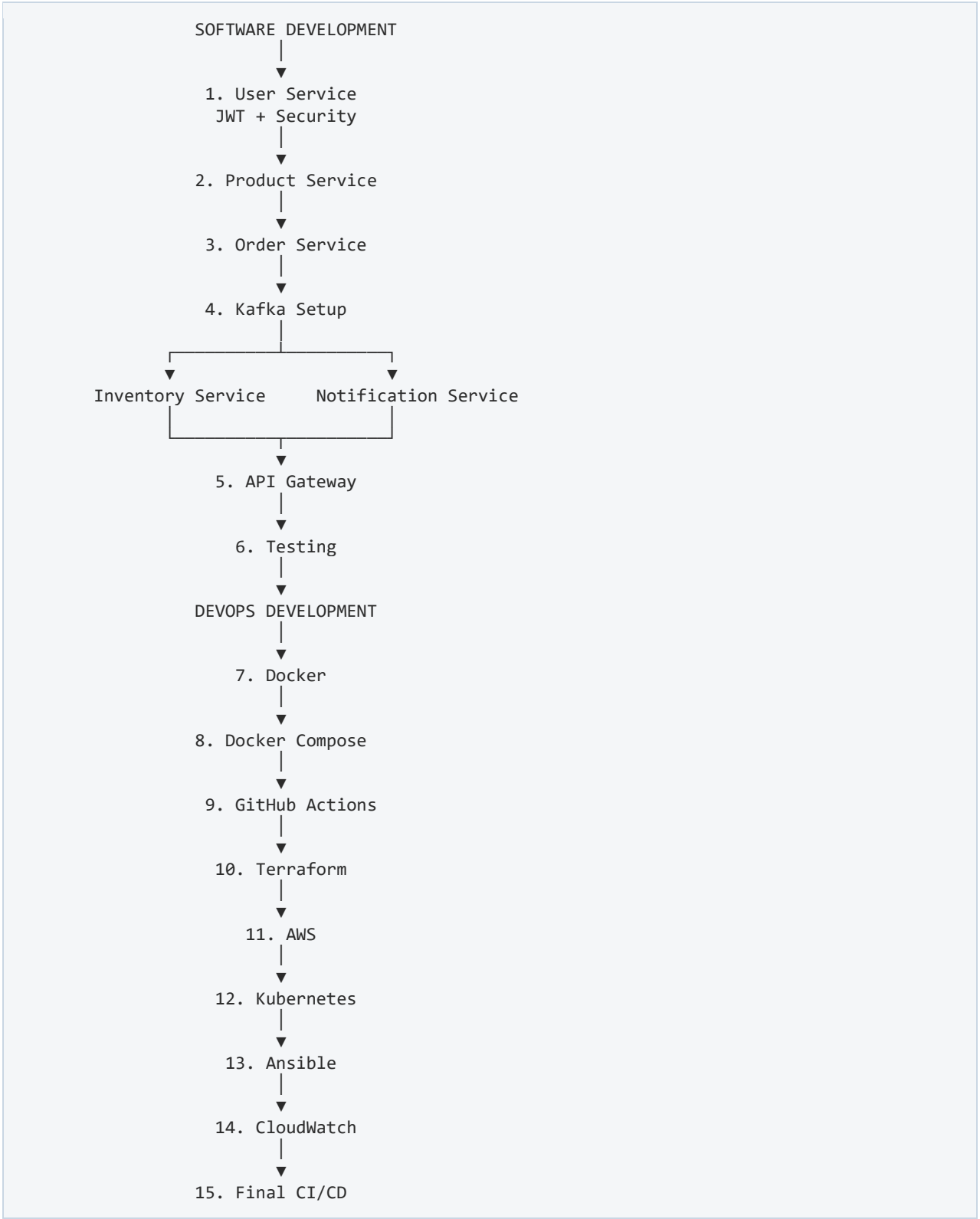
Final monitoring layer:

```
EKS
 ├── Application Logs
 ├── Container Logs
 ├── CPU
 ├── Memory
 ├── Errors
 └── Metrics
       |
       v
   CloudWatch
       |
       v
     Alarm
```

Example alerting flow:

```
Order Service --> CPU 85% --> CloudWatch --> Alarm --> DevOps Engineer
```

Complete Project Roadmap



Immediate Next Step

The User Service scaffold already exists. The immediate priority is to complete the User Service, then proceed in strict order:

Product Service → Order Service → Kafka → Inventory Service → Notification Service → API Gateway

Each of these will be developed with full Spring Initializr settings, package structure, entities, DTOs, controllers, services, repositories, complete implementation code, and Postman testing. Once the application layer is complete, the DevOps phase follows: Docker → CI/CD → Terraform → AWS → Kubernetes → Ansible → CloudWatch.